Data 260

Homework 8

Name: Kshitija Shinde
SJSU ID: 019146491

Value	**Definition**
SID4	6491
PORT_BASE	8000 + (6491 mod 900) = 8000 + 191 = 8191
PREFIX	"s" + 6491 = s6491
SEED	6491
VERIFY_SEED	260000 + 6491 = 266491
DOMAIN_ID	6491 mod 8 = 3
Assigned Domain	Grocery supply and recall notices

Hardware: MacBook Air, 13-inch, M4, 2025, Apple M4 chip, 16GB RAM, macOS Tahoe 26.5.2

Local model used: qwen3:8b via Ollama

Tagged commit hash: 7befc89c2864ce2b48a1635da2dba224b71aecde

Repository: https://github.com/Kshitija-Shinde9/data260-6491

Part 1 - Backend Service Request/Response over Kafka

This homework builds on HW7. Instead of calling your domain logic directly inside one process, you'll split it into a client and a worker that talk only through Kafka messages. This lets you see problems that a single process hides, such as duplicate, lost, or out-of-order messages.

I. Operations

This part must be implemented for your primary domain entity. Replace ENTITY with the actual entity name used by your assigned domain. For example, if your entity is Course, CREATE_ENTITY becomes CREATE_COURSE and entity_id becomes course_id. Replace <PREFIX> with the prefix calculated in Section 0.

Implement the following two operations for your primary domain entity.

• 1. CREATE_ENTITY - Create a new entity
• Input: { "operation": "CREATE_ENTITY", "data": { ...required domain entity fields... } }
• The data object must contain the fields required by your primary domain entity. Do not use user-specific fields unless User is your assigned primary entity.
• Output on success: { "success": true, "entity_id": "generated-uuid", "message": "Entity created" }
• The worker must generate the entity_id using UUID and return it to the client.
• 2. GET_ENTITY - Retrieve an entity by ID
• Input: { "operation": "GET_ENTITY", "data": { "entity_id": "existing-entity-id" } }
• Output on success: { "success": true, "entity": { "entity_id": "...", ...entity fields... } }
• If the requested entity_id does not exist, return success: false with a clear error such as { "success": false, "error": "Entity not found" }.

II. Kafka Requirements

Use exactly two PREFIX-prefixed Kafka topics for this request/response service:

• <PREFIX>_entity_requests - the topic to which the client or API sends requests.
• <PREFIX>_entity_responses - the topic to which the worker sends responses.

Every request message must contain correlation_id, reply_to, operation, and data. The message
must follow this structure:

• { "correlation_id": "unique-id", "reply_to": "<PREFIX>_entity_responses", "operation": "CREATE_ENTITY", "data": { ... } }

The client creates correlation_id once before sending the request. The client, API wrapper, worker, and response must preserve that same value unchanged.

Every response must echo the same correlation_id and include success: true or success: false. A successful response must include the requested entity data or generated entity_id. An error response must include a clean error message.

The worker must:

• Consume messages from <PREFIX>_entity_requests.
• Read the operation and data fields.
• Validate and process CREATE_ENTITY or GET_ENTITY.
• Send the response to the topic named by reply_to.
• Include the unchanged correlation_id in the response.

III. General Requirements

Store entities in a plain Python dictionary inside the worker for this Kafka exercise.

- Generate a unique entity_id with UUID for every successfully created entity.
- Validate all required entity fields and reject missing, empty, or invalid values.
- Reject unknown operations and return a clean error, for example: { "success": false, "error": "Unknown operation" }.
- Return a clean error when GET_ENTITY receives an unknown entity_id.
- Ensure that error responses also contain the original correlation_id.

IV. Demonstration

Create a file named homework_demo.py. It must demonstrate the complete request/response flow in this order:

1. Create one correlation_id.
2. Send a CREATE_ENTITY request to <PREFIX>_entity_requests.
3. Log the message sent to the request topic.
4. Log the worker receiving and processing the request.
5. Log the worker sending the response to <PREFIX>_entity_responses.
6. Log the response received by the client or FastAPI service.
7. Extract the generated entity_id from the CREATE_ENTITY response.
8. Send a GET_ENTITY request using that entity_id.
9. Log the complete GET_ENTITY send, receive, process, reply, and client-response sequence.
10. Display the retrieved entity and confirm that the correlation_id is preserved.

V. Required Screenshots and Evidence

1. Docker containers running: Run docker ps. The screenshot must show the Kafka service, the worker/consumer service, and the FastAPI service running. Include a worker log showing "Consumer is ready and listening..." and a FastAPI log showing the server started at http://127.0.0.1:<PORT_BASE>.

2. CREATE_ENTITY and GET_ENTITY execution: Show the requests and successful responses from
homework_demo.py or an API call. The evidence must show the operation, entity data, generated entity_id, success status, and correlation_id.

3. Kafka message-flow logs: Show the message sent to <PREFIX>_entity_requests, the message
received and processed by the worker, the response sent to <PREFIX>_entity_responses, and the response received by FastAPI or the client and returned to the caller.

4. Full sequence: Submit a log excerpt showing CREATE_ENTITY -> GET_ENTITY with the send,
receive, process, reply, and final client-response lines in order.
Submit screenshot of your Kafka client, homework_demo.py, worker, and API wrapper if used. Your report must show the two topic names, the request and response message schemas, the complete correlation_id flow, and the required screenshots.

Part 1

Shapes every message I send.

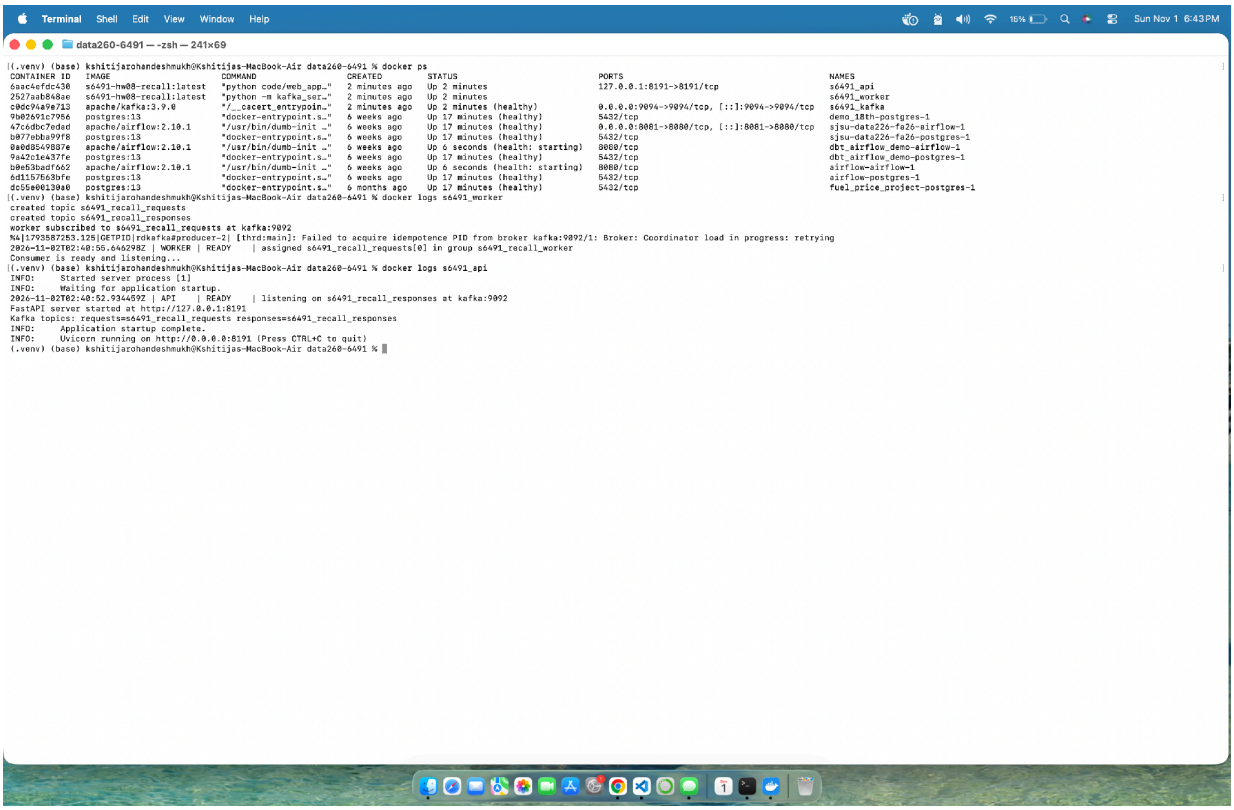
code/kafka_service/kafka_client.py

```
def build_request(operation: str, data: dict, correlation_id: str) -> dict:
    return {
        "correlation_id": correlation_id,
        "reply_to": RESPONSE_TOPIC,
        "operation": operation,
        "data": data,
    }
```

Screenshot 1. The three programs are running.

code/kafka_service/worker.py and kafka_api.py

```
print("Consumer is ready and listening...", flush=True)
print(f"FastAPI server started at http://127.0.0.1:{PORT_BASE}", flush=True)
print(f"Kafka topics: requests={REQUEST_TOPIC} responses={RESPONSE_TOPIC}",
flush=True)
```

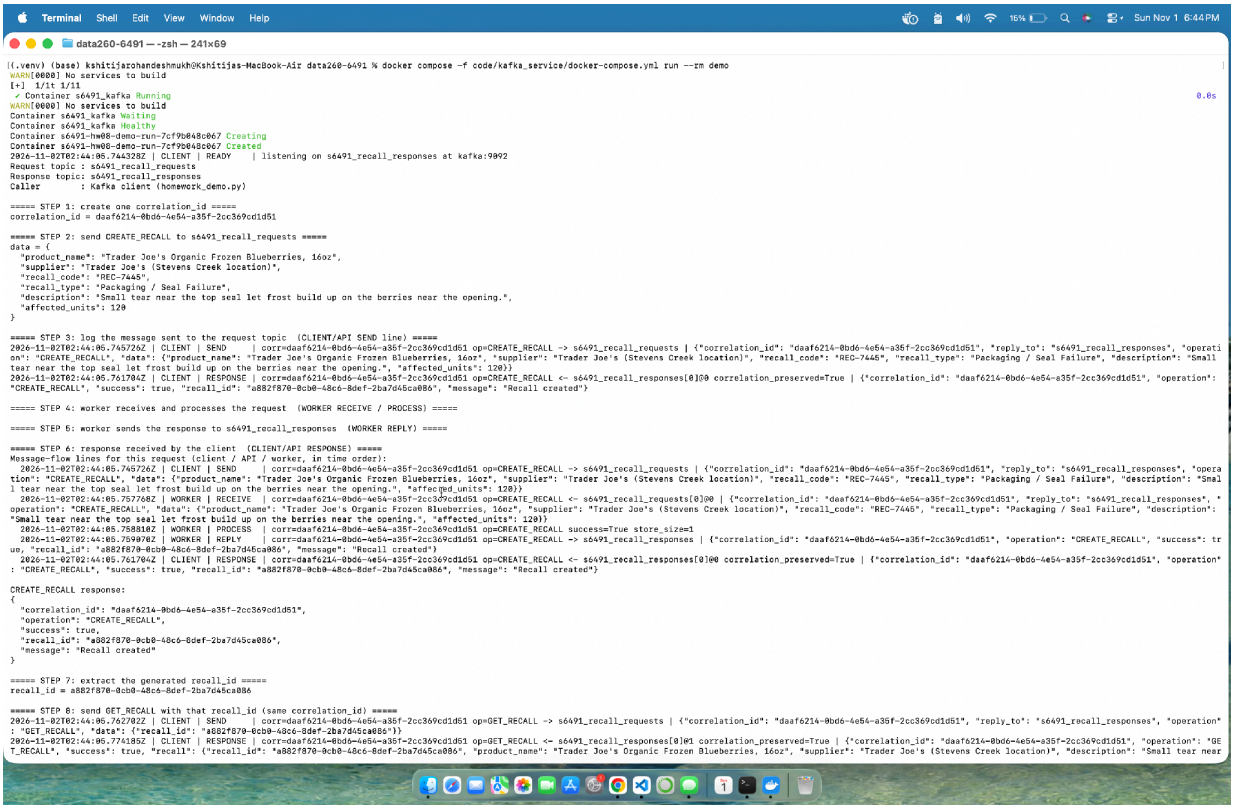


Screenshot 2a. Creating a recall from the demo.

Lines in the middle are the worker picking the message up and sending the answer back.

code/kafka_service/homework_demo.py

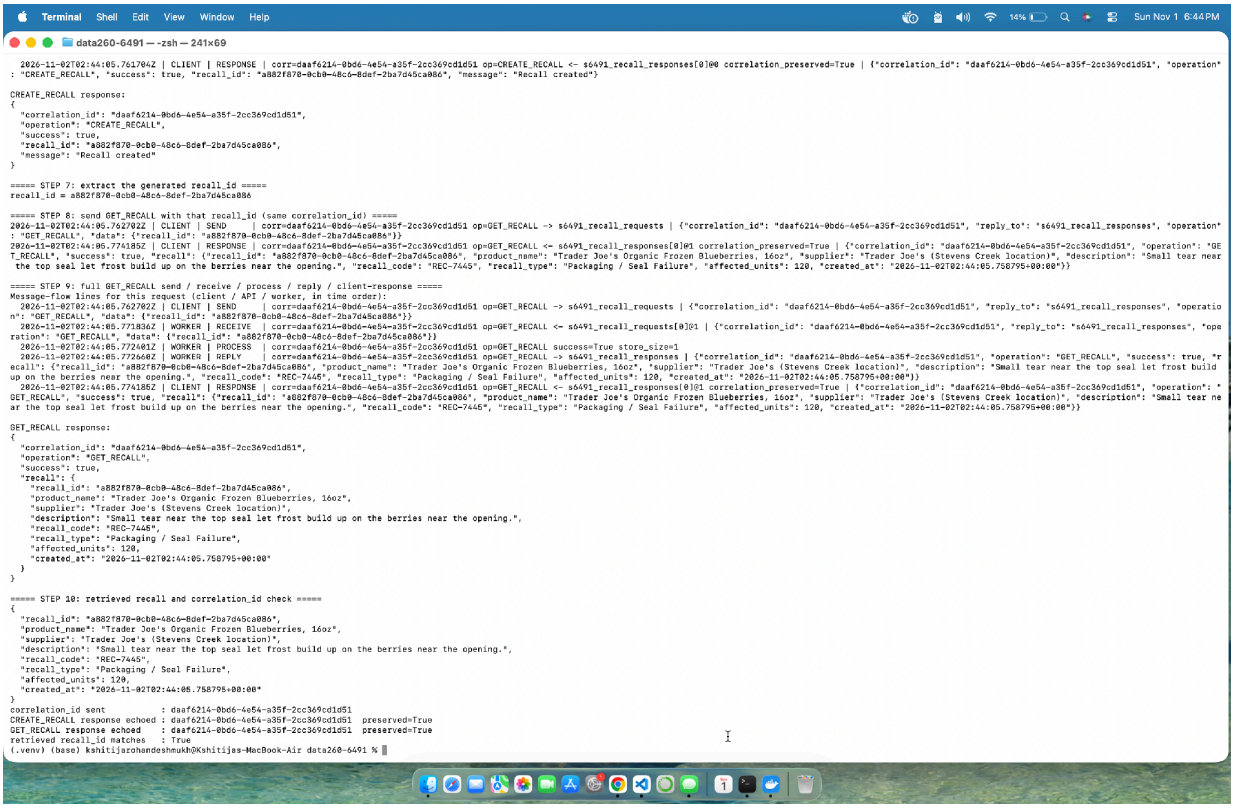
```
        correlation_id = new_correlation_id()
        print(f"correlation_id = {correlation_id}")
        recall_data = sample_recall()
        create_response = caller.call(CREATE_OPERATION, recall_data, correlation_id)
        recall_id = create_response["recall_id"]
```



Screenshot 2b. Looking that recall back up.

code/kafka_service/homework_demo.py

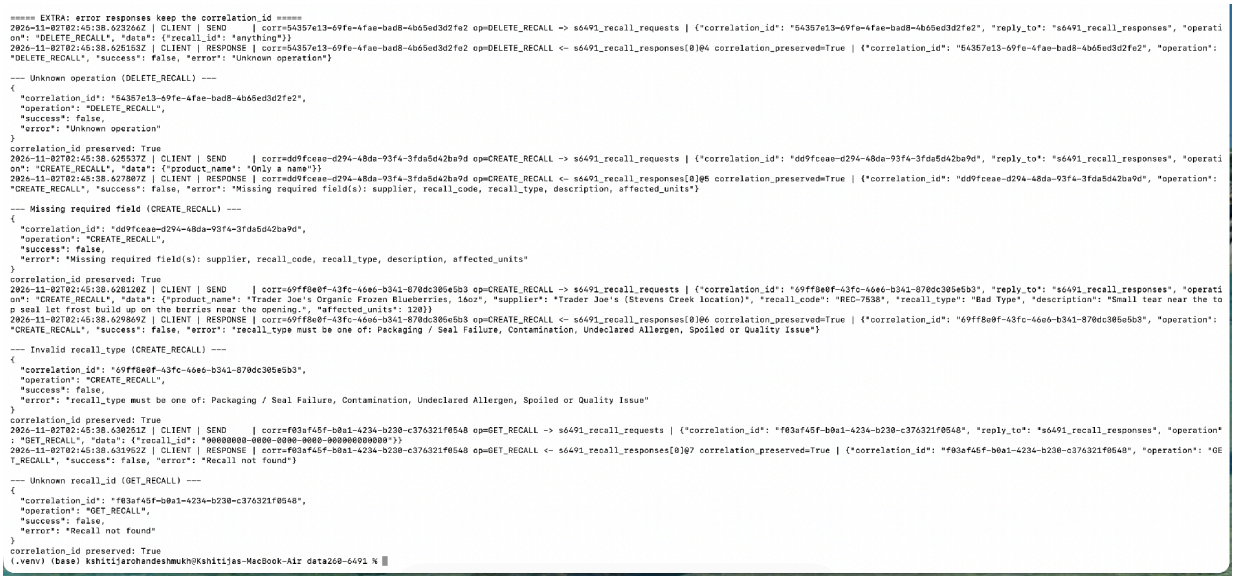
```
get_response = caller.call(GET_OPERATION, {"recall_id": recall_id}, correlation_id)
```



Screenshot 2c. Bad requests still bring the same id back.

code/kafka_service/worker.py

```
def create_recall(data: Any, store: dict) -> dict:
    fields, error = validate_recall_fields(data)
    if error:
        return {"success": False, "error": error}
    if any(record["recall_code"] == fields["recall_code"] for record in
store.values()):
        return {"success": False, "error": f"recall_code {fields['recall_code']}
already exists"}
    recall_id = str(uuid.uuid4())
    store[recall_id] = {
        "recall_id": recall_id,
        **fields,
        "created_at": datetime.now(timezone.utc).isoformat(),
    }
    return {"success": True, "recall_id": recall_id, "message": "Recall created"}
```



Screenshot 3a. The same two calls, this time through the web API.

code/web_application/routers/kafka_recalls.py

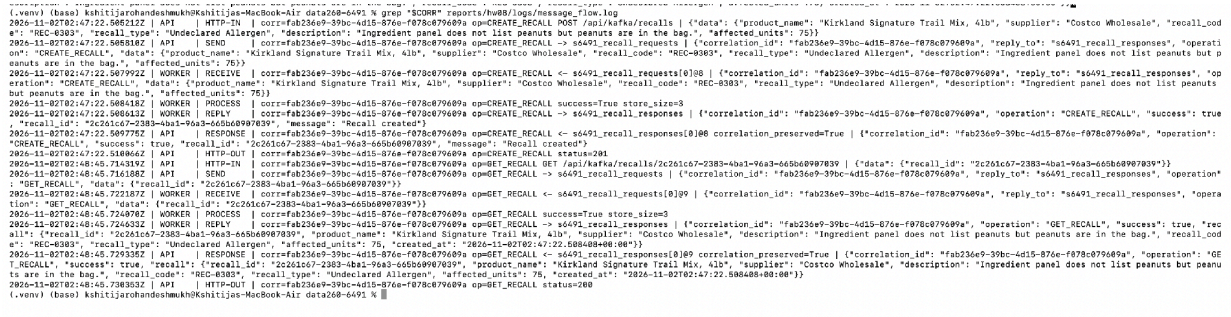
```
@router.post("/recalls")
def create_recall_over_kafka(data, x_correlation_id=None):
    return forward_to_worker("CREATE_RECALL", data, x_correlation_id,
success_status=201)
@router.get("/recalls/{recall_id}")
def get_recall_over_kafka(recall_id, x_correlation_id=None):
    return forward_to_worker("GET_RECALL", {"recall_id": recall_id}, x_correlation_id)
```



Screenshot 3b. The whole trip, written into one log.

code/web_application/routers/kafka_recalls.py

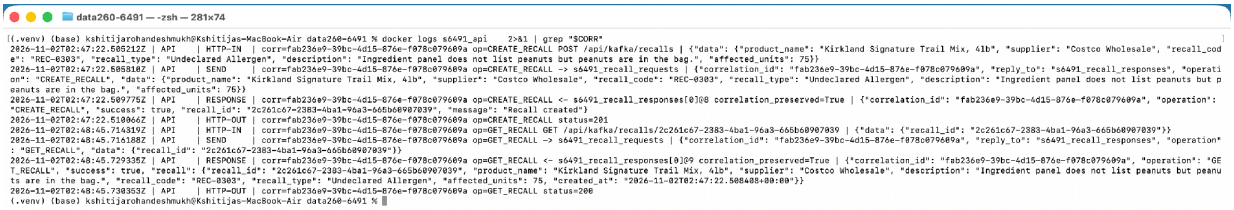
```
log_flow("API", "HTTP-IN", f"corr={correlation_id} op={operation} {http_route}")
response = get_kafka_client().request(operation, data, correlation_id=correlation_id)
log_flow("API", "HTTP-OUT", f"corr={correlation_id} op={operation}
status={status_code}")
```



Screenshot 3c. The API's own log for that same call.

code/web_application/routers/kafka_recalls.py

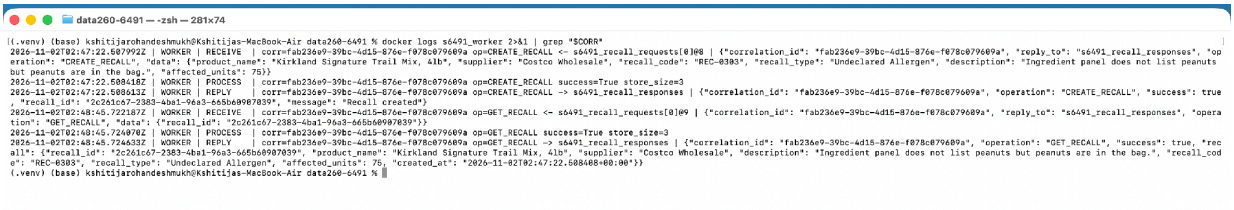
```
correlation_id = (correlation_id or "").strip() or new_correlation_id()
response = get_kafka_client().request(operation, data, correlation_id=correlation_id)
return JSONResponse(status_code=status_code, content=response,
                    headers={"X-Correlation-ID": correlation_id})
```



Screenshot 3d. The worker's own log for that same call.

code/kafka_service/worker.py

```
log_flow("WORKER", "RECEIVE", f"corr={correlation_id} op={operation}")
reply_topic, response = handle_request(message, store)
log_flow("WORKER", "PROCESS", f"corr={correlation_id} success={response['success']}")
log_flow("WORKER", "REPLY", f"corr={correlation_id} -> {reply_topic}")
send_json(producer, reply_topic, correlation_id, response)
```

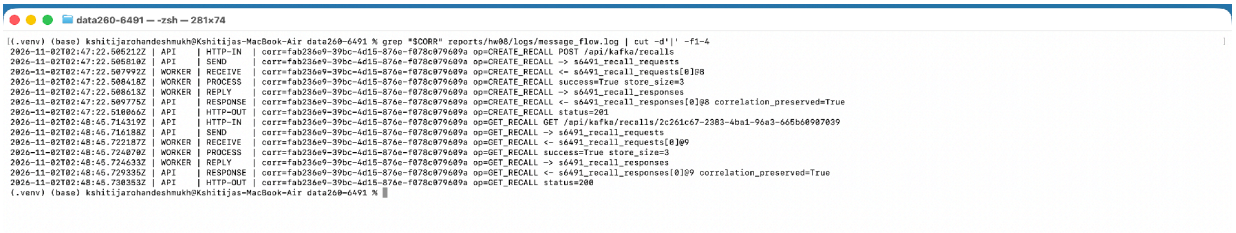


Screenshot 4. Create, then lookup, one step per line.

worker.py and kafka_recalls.py

```
log_flow("API", "HTTP-IN", f"corr={correlation_id} op={operation}")
```

```
log_flow("API", "SEND", f"corr={correlation_id} -> s6491_recall_requests")
log_flow("WORKER", "RECEIVE", f"corr={correlation_id} op={operation}")
log_flow("WORKER", "PROCESS", f"corr={correlation_id} success={response['success']}")
log_flow("WORKER", "REPLY", f"corr={correlation_id} -> s6491_recall_responses")
log_flow("API", "RESPONSE", f"corr={correlation_id} op={operation}")
log_flow("API", "HTTP-OUT", f"corr={correlation_id} status={status_code}")
```



Part 2 - Three-Agent Kafka Pipeline

Run Planner, Writer, and Reviewer as separate processes. No agent calls another agent directly; every exchange goes through Kafka.

I. Topics and Workflow Use four PREFIX-prefixed topics: <PREFIX>_inbox, <PREFIX>_tasks, <PREFIX>_drafts, <PREFIX>_final.

Sender	Reads Topic	Sends Message To
You	-	<PREFIX>_inbox (a question)
Planner	<PREFIX>_inbox	<PREFIX>_tasks (a plan)
Writer	<PREFIX>_tasks	<PREFIX>_drafts (a draft answer)
Reviewer	<PREFIX>_drafts	<PREFIX>_final (an approval)

II. Kafka Topic Creation Use your Kafka CLI or UI to create the four topics. Example CLI commands:

kafka-topics --create --topic <PREFIX>_inbox --partitions 1 --replication-factor 1 --bootstrap-server localhost:9092

kafka-topics --create --topic <PREFIX>_tasks --partitions 2 --replication-factor 1 --bootstrap-server localhost:9092

kafka-topics --create --topic <PREFIX>_drafts --partitions 1 --replication-factor 1 --bootstrap-server localhost:9092

kafka-topics --create --topic <PREFIX>_final --partitions 1 --replication-factor 1 --bootstrap-server localhost:9092

III. Running the Agents

Run each agent in its own terminal:

Terminal	Command
Terminal 1	python planner.py
Terminal 2	python writer.py
Terminal 3	python reviewer.py
Terminal 4	python send_question.p

IV. Result

The final message is JSON with status and answer. Preserve correlation_id through every stage. Read the result from <PREFIX>_final.

Submit:

- Your Planner, Writer, and Reviewer code.
- One complete message journey from <PREFIX>_inbox to <PREFIX>_final, shown as log lines for each stage.
- The final JSON message with its status and answer.
- A short report explaining how Kafka helped your agents coordinate.

Part 2

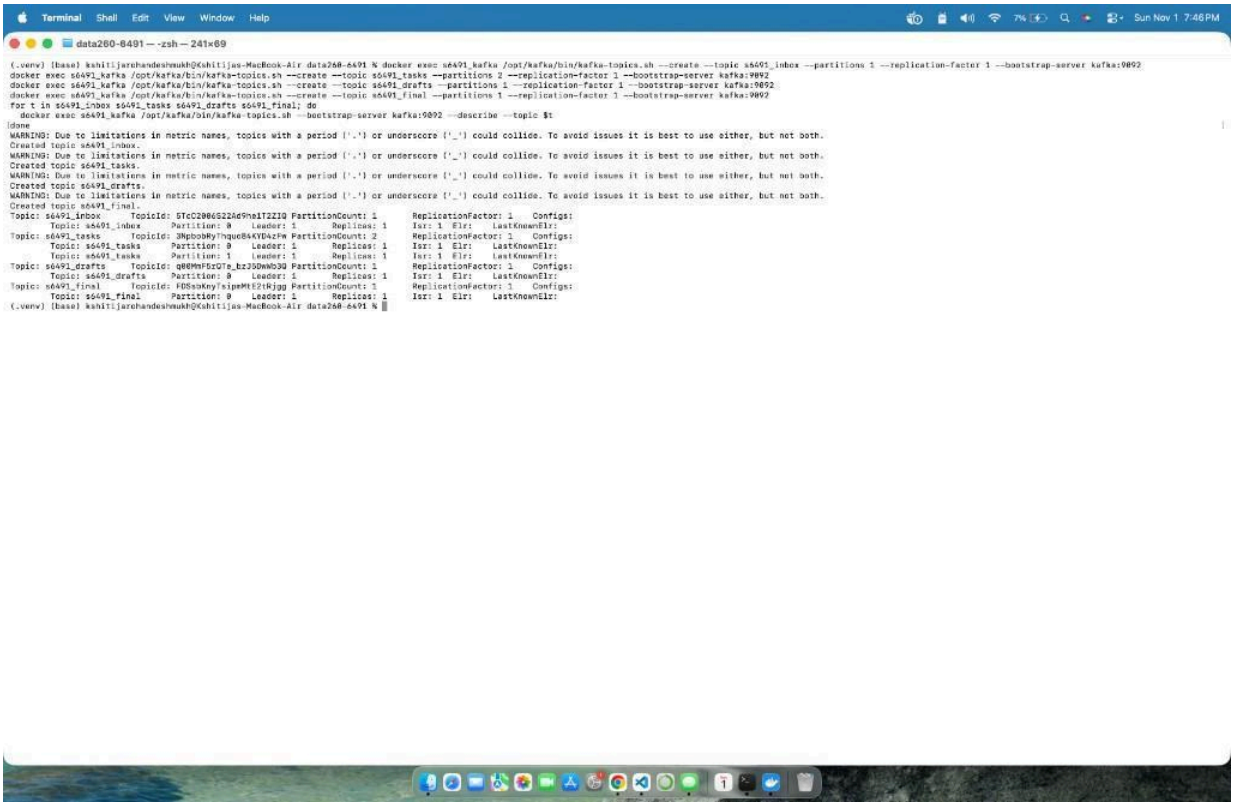
code/kafka_service/reviewer.py

```
def handle(message):
    status, answer = review_draft(str(message.get("draft", "")))
    return {
        "correlation_id": message["correlation_id"],
        "status": status,
        "answer": answer,
    }
```

Screenshot A. Making the four channels.

code/kafka_service/config.py

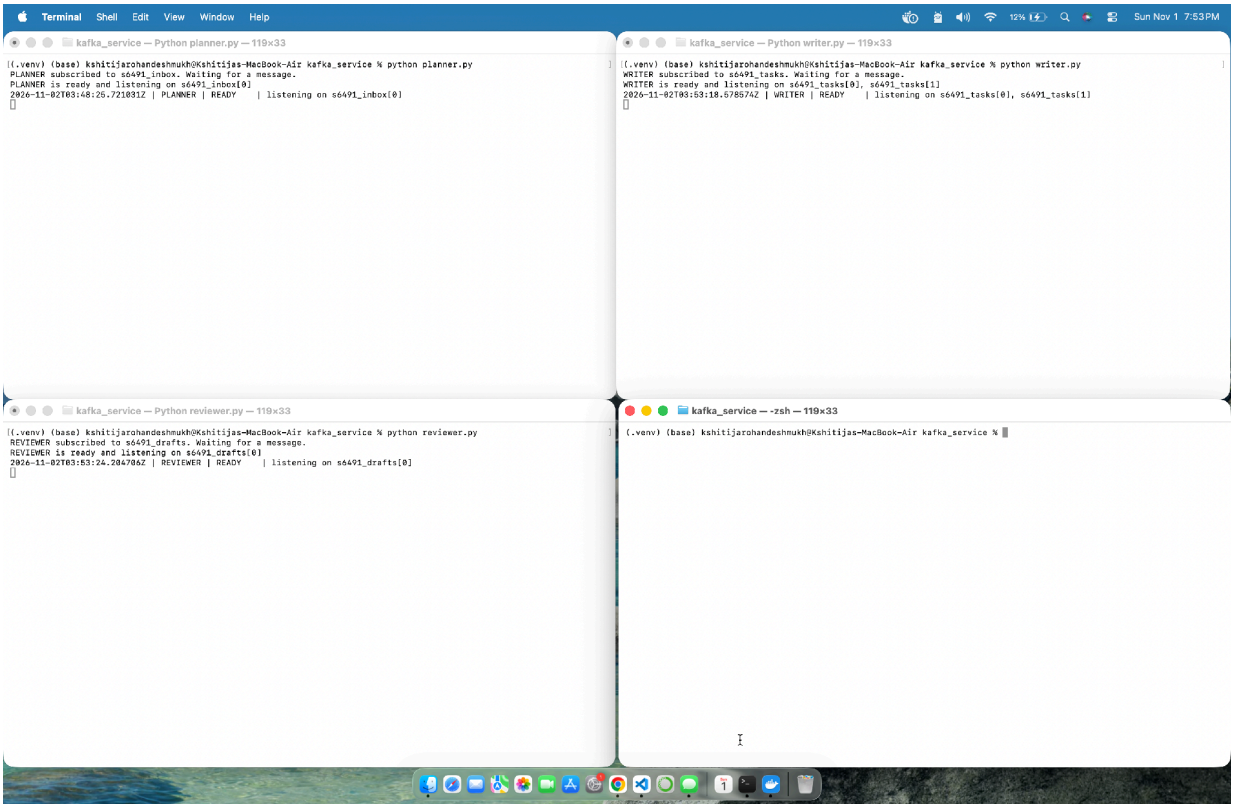
```
INBOX_TOPIC = f"{PREFIX}_inbox"
TASKS_TOPIC = f"{PREFIX}_tasks"
DRAFTS_TOPIC = f"{PREFIX}_drafts"
FINAL_TOPIC = f"{PREFIX}_final"
PART2_TOPICS = (
    (INBOX_TOPIC, 1),
    (TASKS_TOPIC, 2),
    (DRAFTS_TOPIC, 1),
    (FINAL_TOPIC, 1),
)
```



Screenshot B. Three agents, three terminals, all waiting.

planner.py, writer.py and reviewer.py

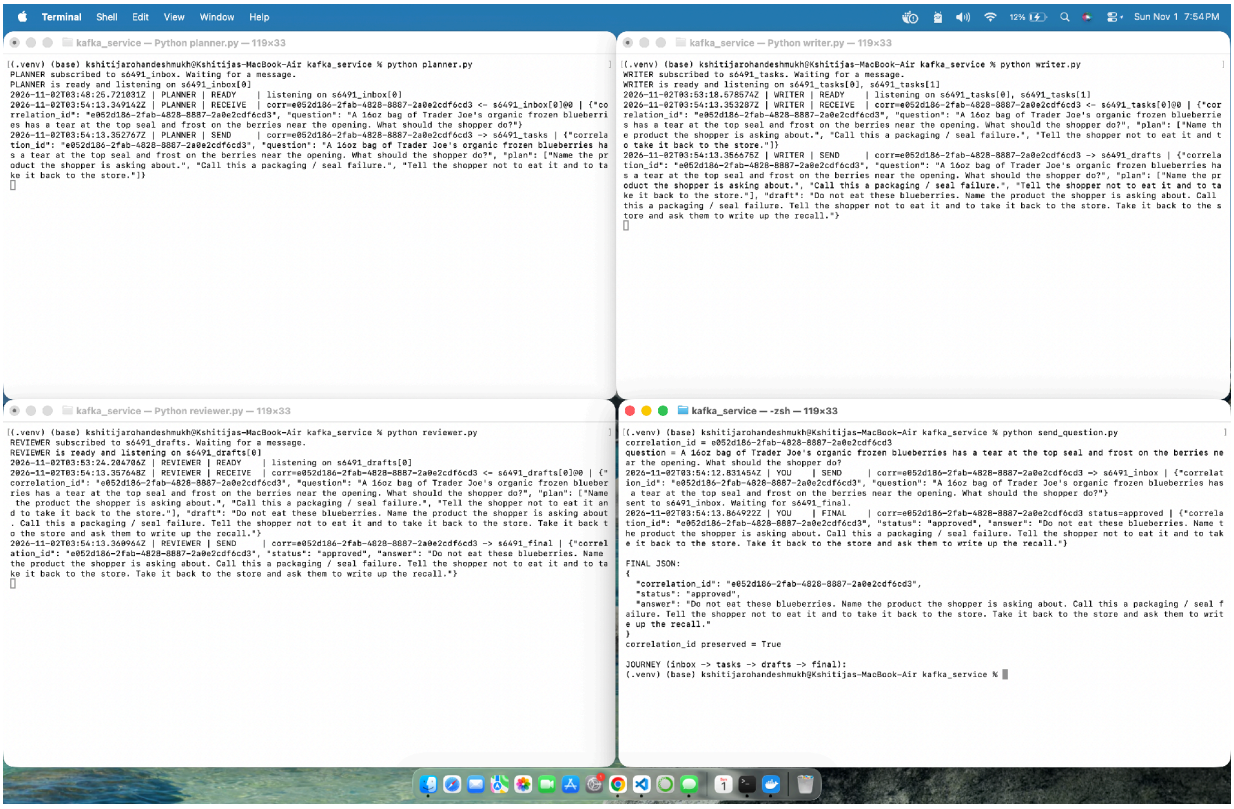
```
# planner.py
serve("PLANNER", INBOX_TOPIC, TASKS_TOPIC, "s6491_planner", handle)
# writer.py
serve("WRITER", TASKS_TOPIC, DRAFTS_TOPIC, "s6491_writer", handle)
# reviewer.py
serve("REVIEWER", DRAFTS_TOPIC, FINAL_TOPIC, "s6491_reviewer", handle)
```



Screenshot C. One question, passed from agent to agent.

code/kafka_service/pipeline.py

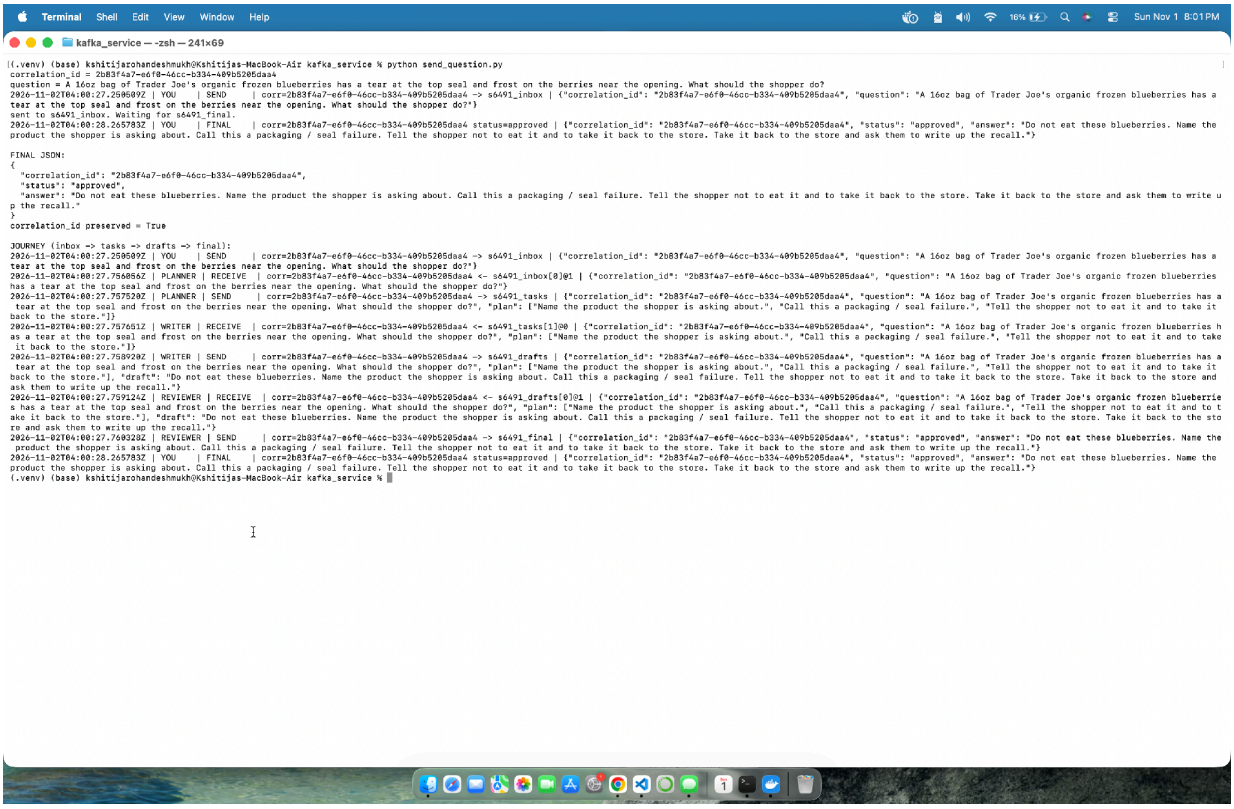
```
correlation_id = incoming["correlation_id"]
log_flow(component, "RECEIVE", f"corr={correlation_id} <- {location}", incoming)
outgoing = handle(incoming)
outgoing["correlation_id"] = correlation_id
send_json(producer, target, correlation_id, outgoing)
log_flow(component, "SEND", f"corr={correlation_id} -> {target}", outgoing)
```



Screenshot D. The final answer and the full journey in one place.

code/kafka_service/send_question.py

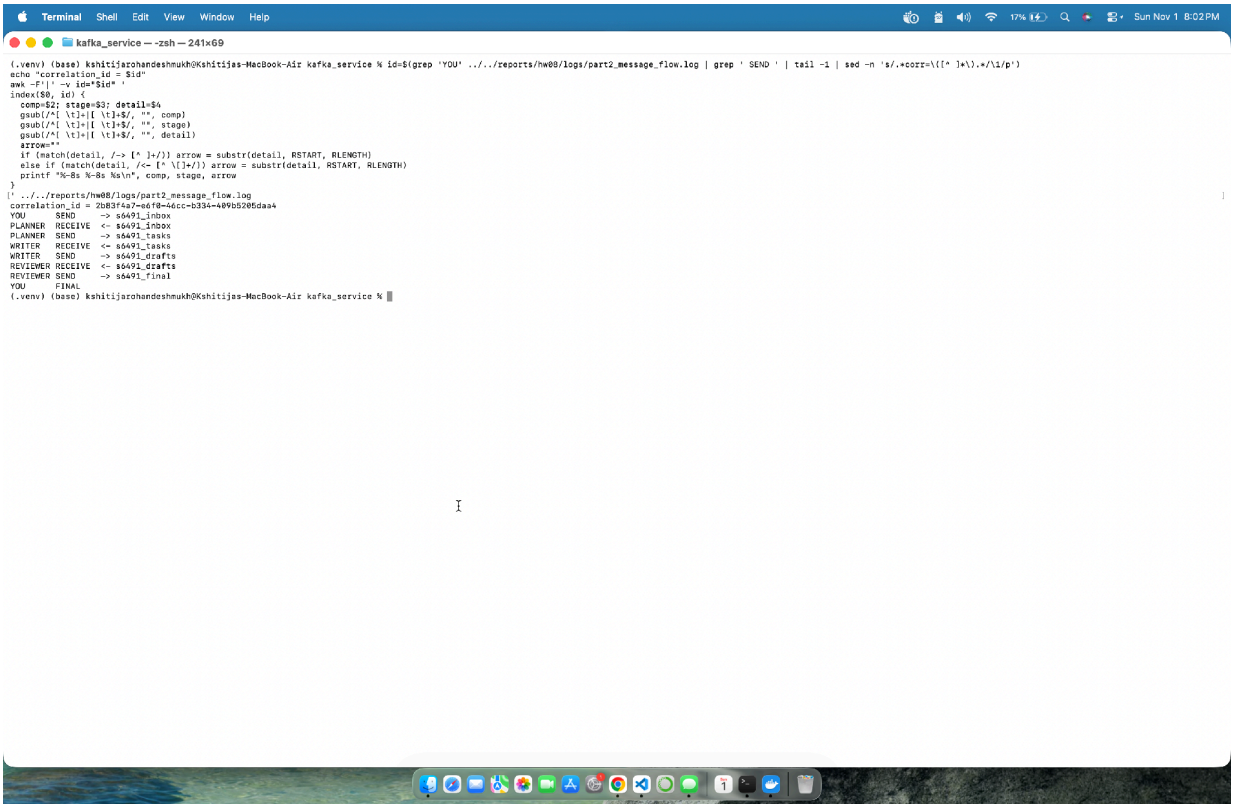
```
message = {"correlation_id": correlation_id, "question": question}
send_json(producer, INBOX_TOPIC, correlation_id, message)
final = waiter.wait(correlation_id, timeout_sec=30)
kept = final.get("correlation_id") == correlation_id
print(f"correlation_id preserved = {kept}")
```



Screenshot E. The same journey, one step per line.

code/kafka_service/flow_log.py

```
message = {"correlation_id": correlation_id, "question": question}
send_json(producer, INBOX_TOPIC, correlation_id, message)
final = waiter.wait(correlation_id, timeout_sec=30)
kept = final.get("correlation_id") == correlation_id
print(f"correlation_id preserved = {kept}")
```



The three agents, in short

Each agent is a small function that gets one message and returns the next one. The shared loop in pipeline.py does the Kafka part, so the agents themselves never touch another agent.

code/kafka_service/planner.py

```
plan = ["Name the product the shopper is asking about."]

if any(word in text for word in ("seal", "tear", "frost", "torn")):

   plan.append("Call this a packaging / seal failure.")

plan.append("Tell the shopper not to eat it and to take it back to the store.")
```

code/kafka_service/writer.py

```
return (

   f"Do not eat {product}. {steps} "

   "Take it back to the store and ask them to write up the recall."

)
```

code/kafka_service/reviewer.py

```
text = (draft or "").strip()

if text and "do not eat" in text.lower():

   return "approved", text

if not text:

   return "rejected", "No answer was written."
```

How Kafka helped my agents coordinate

The planner, the writer and the reviewer never talk to each other directly. I only put a question on the inbox channel. Whoever is listening picks it up, does its one job, and puts the result on the next channel. Kafka is the only thing connecting them.

That made a few things easier. I could start the three agents in any order, because each one just waits on its own channel until something shows up. If one of them stops, the messages wait in Kafka instead of getting lost, and the agent picks up where it left off when it comes back. The tasks channel has two partitions, so if I started a second writer, Kafka would split the work between them without me changing any code.

The correlation id is what ties it all together. It rides along on every message, so when an answer shows up on s6491_final I know exactly which question it belongs to, even if several questions are moving through at the same time.

Part 3: Failure Behavior

I. Duplicate Delivery Replay a message with the same correlation_id, then add idempotency so replay causes no extra side effect.

II. Consumer Death Terminate Writer mid-processing and record what happens. After restarting Writer, show that the message is redelivered or completed, and ensure the offset is committed only after successful processing.

III. Poison Message

Add bounded retries and a PREFIX-prefixed dead-letter topic named <PREFIX>_dead_letter. Show that a bad message is retried up to the limit, moved to the dead-letter topic, and no longer blocks later messages.

IV. Ordering

Using VERIFY_SEED, generate 20 related messages (for example, several about the same entity). Send them through <PREFIX>_tasks configured with two partitions and two Writer instances in the same consumer group, and record the order in which they are processed. Then choose a message key based on the related entity, re-run the test, and show that all related messages stay in one partition and preserve their order.

V. Comparison

In HW2, your graph ran as function calls within one process, where one step waited on the previous one directly. Compare that with this Kafka-based design: name two benefits you gained by moving to Kafka (for example, independent scaling or decoupled failure) and two new responsibilities you now have to handle yourself that you didn't before (for example, ordering and duplicate delivery).

	HW2, in-process graph	HW8 Part 3, Kafka
Benefit: scaling	One planner step waited on a function call. Adding more workers meant changing the code.	I ran a second writer and Kafka split s6491_tasks between them by partition, with no code change.
Benefit: surviving a crash	If the process died mid-step, the whole chain died with it. Nothing was waiting anywhere.	When the writer was killed mid-message, the message stayed in Kafka. A restart picked it up and finished it.
New job: ordering	A function call happens, then the next one happens. Order was automatic.	Two partitions do not keep one product in order by themselves. I had to key messages by product name myself.
New job: duplicates	A function runs once, because I only called it once.	Kafka can redeliver a message. The writer has to remember the correlation_id itself or it writes two drafts.

Fill in the failure table with before/after evidence for each row:

Failure scenario	Before fix	After fix
Duplicate delivery	No idempotency. dup-266491 sent twice, the writer ran handle() twice and produced two SEND lines, so there were two drafts for one question.	Idempotency on. dup-266491 sent twice, one SEND for the first copy, then DUPLICATE and "already handled, no second draft" for the second copy.
Consumer killed mid-work	COMMIT happened before the draft was sent, then CRASH. On restart the writer only showed READY, no RECEIVE. redelivered = no.	CRASH happened before any COMMIT. On restart the writer did RECEIVE, SEND, COMMIT for the same id. redelivered = yes.
Poison message	No retry limit and no dead-letter topic. poison-266491 would keep failing forever, so good-266491 behind it would never get a draft.	3 attempts, then DEAD to s6491_dead_letter. poison-266491 landed there with attempts = 3, and good-266491 was drafted right after it.
Two partitions/two consumers	No product key. With 20 messages across 4 products, one product's updates landed on both partitions and came out in a different order than they were sent.	Key = product name. Every product's updates stayed on one partition, in the exact order they were sent.

Submit:

- The completed failure table above, with a matching log excerpt for every row.
- Your idempotency, offset-commit, retry/dead-letter, and partition-key code.
- Your matching logs must show the duplicate replay result, the consumer restart and offset behavior, the retry and <PREFIX>_dead_letter result, and both ordering runs.
- Your two-benefits/two-responsibilities comparison with the HW2 in-process graph

Part 3

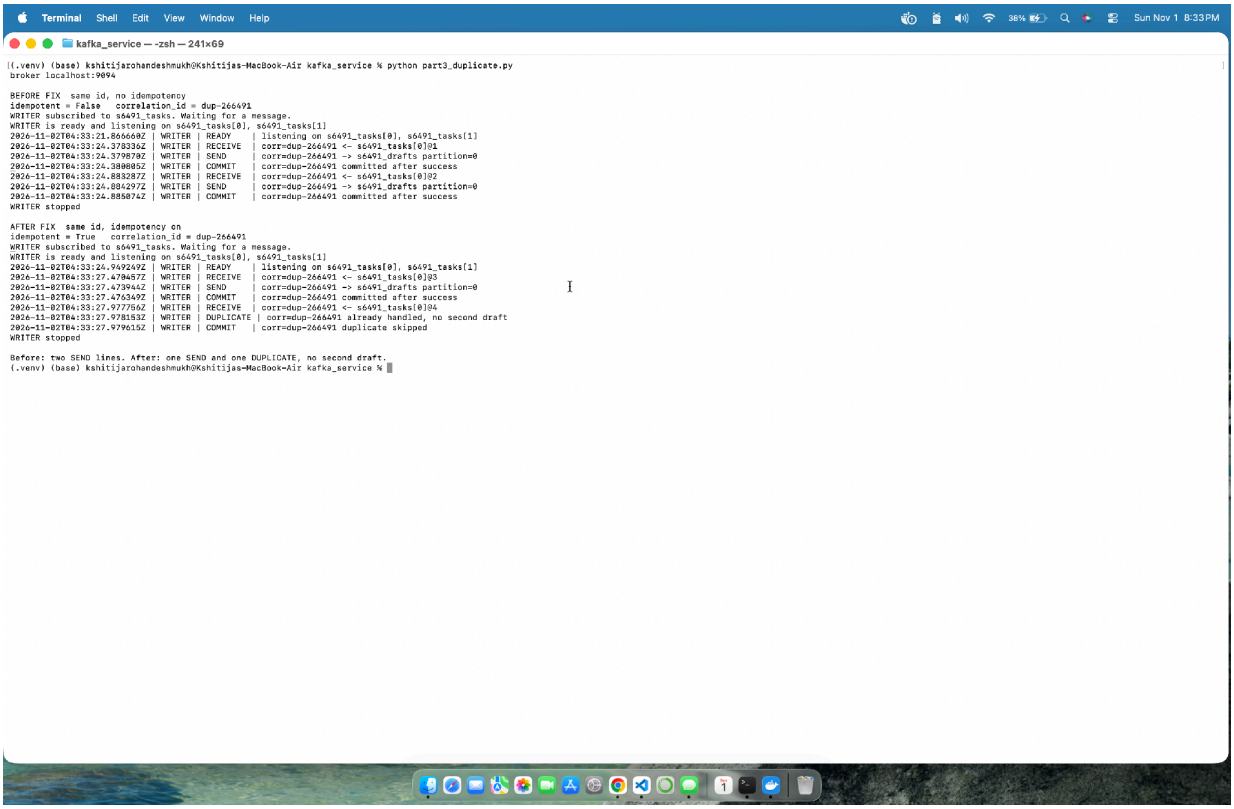
code/kafka_service/failure.py

```
def decide(message, seen, idempotent, handle, max_attempts, on_retry=None):
    correlation_id = message["correlation_id"]
    if idempotent and correlation_id in seen:
        return {"action": "duplicate", "outgoing": seen[correlation_id]}
```

Screenshot 1. Sending the same question twice.

code/kafka_service/pipeline.py

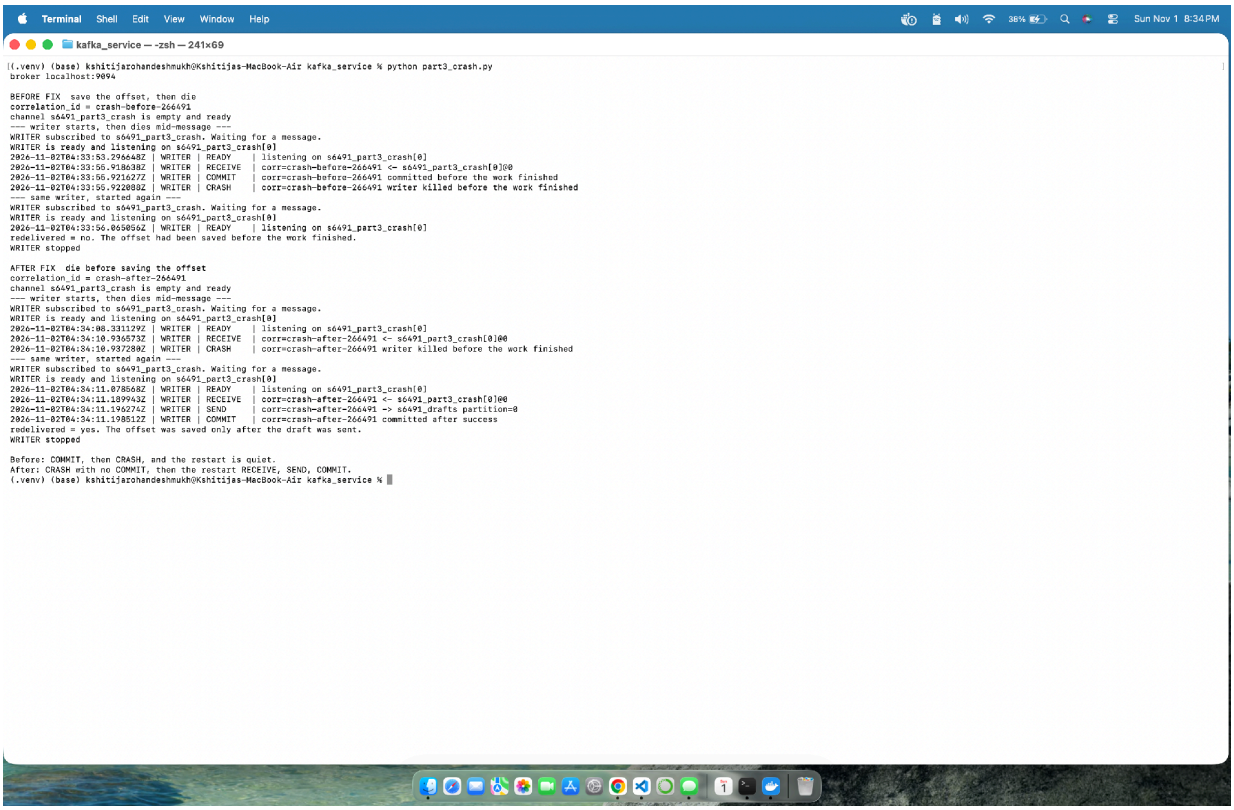
```
if idempotent and correlation_id in seen:
    log_flow(component, "DUPLICATE", f"corr={correlation_id} already handled, no second draft")
    _commit(consumer, msg, component, correlation_id, "duplicate skipped")
```



Screenshot 2. Killing the writer in the middle of a message.

code/kafka_service/pipeline.py

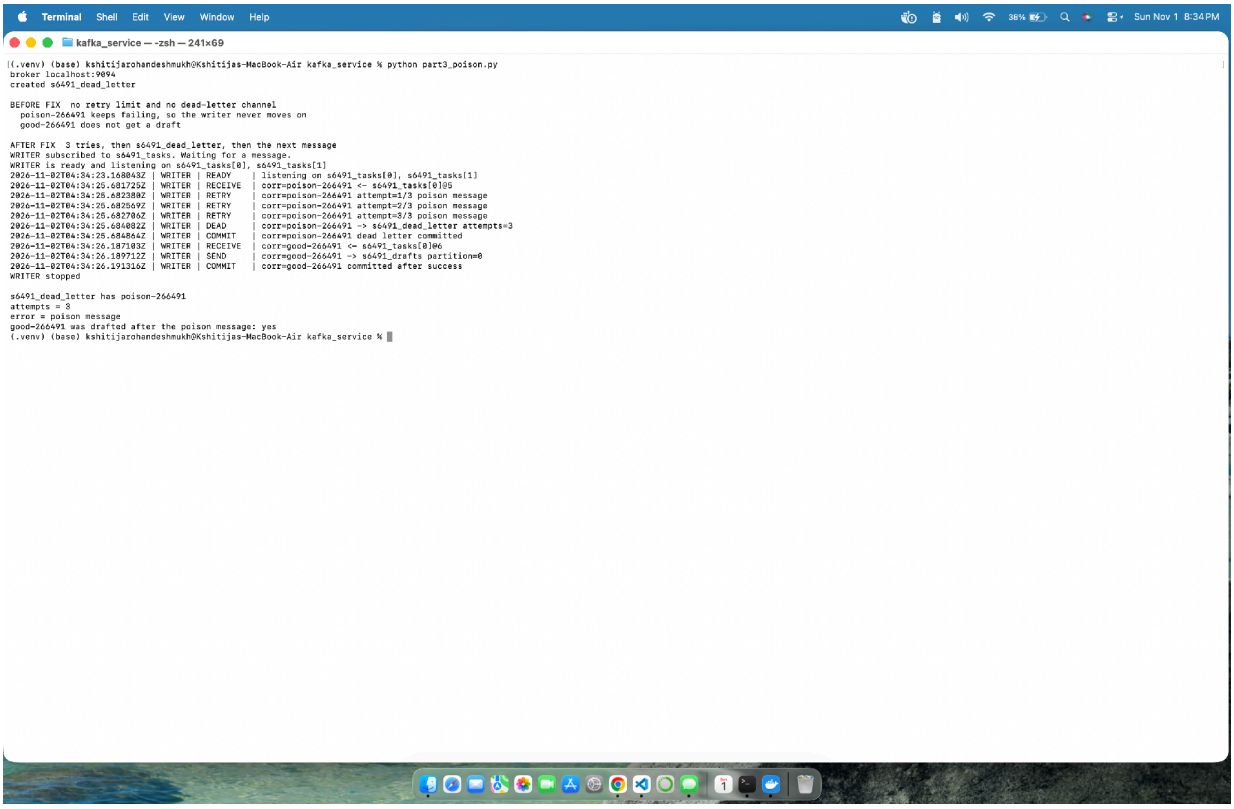
```
if 'EARLY_COMMIT' in steps:
    _commit(consumer, msg, component, correlation_id, "committed before the work finished")
if 'CRASH' in steps:
    _crash(consumer, producer, component, correlation_id)
```



Screenshot 3. A bad message that will never succeed.

code/kafka_service/writer.py

```
def handle(message):
    if message.get("poison"):
        raise PoisonMessage("poison message")
```



Screenshot 4. Keeping one product's updates in order.

code/kafka_service/part3_ordering.py

```
if keyed:
    publish(TASKS_TOPIC, message, key=message["entity"])
else:
    publish(TASKS_TOPIC, message, key=None, partition=index % 2)
```



How Kafka helped, and what it cost me

Moving this work onto Kafka gave me two things I did not have when the planner and reviewer were plain function calls in one process. I could add a second writer and Kafka split the work between them without me touching the writer's code. And when a writer died partway through a message, the message just sat in Kafka until a writer came back to pick it up, instead of being lost with the process.

It also gave me two new jobs I did not have before. Kafka does not promise that one product's updates stay together unless I pick a partition key myself, so I had to key messages by product name to keep their order. And Kafka can hand a writer the same message twice, so the writer has to remember what it already did instead of trusting that each message only arrives once.